DATA ANALYTICS PROJECT

Railway Gauges

Understanding Railway Gauge Trends: From Raw Data to Real Insights.

This document gives us a structured and complete understanding of data analysis through railway gauge data.

Project Goal: To analyse the historical trends and comparison of different railway gauges (Broad, Metre, and Narrow) over years**.**

Dataset Used: railway_gauges1.csv

Packages/Modules/Library used: The following are imported at the start of the project.

- **NumPy:** For all mathematical Calculations.
- **Pandas:** For all data manipulations, filtering, aggregation and featuring & engineering tasks.
- **Visualization Library (Matplotlib):** For visualizing the graphs.

What does this Project cover?

- **Data Pre-processing:** Data Loading, Handling the Missing Values, Type Conversions.
- **Pandas Operations:** Filtering, Selection, Aggregation, Feature Engineering.
- **NumPy Calculations:** All the calculations done using numpy.
- **Visualization:** Line Graphs, Bar Graphs, Pie Charts, Stacked Bar Chart.

In order to move forward in the code part,

Firstly, we need to import the required packages and modules like “Numpy”, “Pandas”, and “Matplotlib” in the VS code or the platform where the code is written. We also need to import pyplot from matplotlib in order for plotting the required graphs.

```
#Lets import the required packages and modules
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
```

Now let’s Move to the Scenarios:

Scenario 1: Basic Data Loading and Cleaning

1: Load the Dataset into Pandas DataFrame

Here after importing the modules, we need to now load the data and process it. In pandas “read_csv()” is used to read the csv file, so we used the function that is defined below.

```
#Loading the .Csv file to process the data
df=pd.read_csv('railway_gauges1.csv')
df.columns = df.columns.str.strip()
```

And in the figure we can see we used strip() function, it is important because in certain files like .csv or Excel files we may have hidden spaces, that may raise “Key Error”. So, this function removes hidden spaces, not raising the errors.

2: Display the First 5 Rows and Columns

After loading the data now let's process the first 5 rows and then column names. In Pandas we have a function called as "head()" that gives the first 5 rows data.so we use print(df.head()) to display the first 5 rows data in columns. And for column name we have "df.columns" which displays the column names that are present.

```
#Processing the first 5 rows and colunms names
print("First Five ROWS ")
print(df.head())#df.head() gives the first 5 frowswi
#column Names
print("The COLUMN Names")
print(df.columns)
```

3. Check for the Missing Values and Replace them with 0.

Here, "isnull()" function checks the data for empty values i.e. it scans every cell for any empty value, and if found empty it returns "True" else it returns "False". The "sum()" counts the total number of missing values in each column, which gives us a clear understanding where the values are empty. Next we have "filna(0)", this function fills the empty spaces with 0 in the cells.

```
#Check for missing values and replace them with 0.
#it checkes the data is null or not  and if null it
print(df.isnull().sum())
df=df.fillna(0)
#print(df)
```

4. Convert all the Gauge Columns to Numeric Type

Here we first defined a list of the columns, and then we loop each one to convert them to numeric through pd.to_numeric()(- a function to convert the data type to numeric). And as we know if the data contains any non-numeric values that throws an error. So, "coerce" is used to handle the error, which converts error into a "NAN". Then later in the line we have fillna(0),that will replace "NAN" with "0" without stopping the flow of the code.

```
#Convert all gauge columns (Broad, Metre, Narrow, Total) to numeric types.
gauge_columns = ['Broad Gauge', 'Metre Gauge', 'Narrow Gauge', 'Total']
for col in gauge_columns:
    df[col]=pd.to_numeric(df[col],errors='coerce').fillna(0)
print("\nData types after conversion:")
print(df.dtypes)
```

O\P Observed are:

```
First Five ROWS
      Year  ...  Total
0  1964-65  ...   6876
1  1965-66  ...   6986
2  1966-67  ...   7007
3  1967-68  ...   7027
4  1968-69  ...   7032
```

```
Data types after conversion:
Year              str
Broad Gauge     int64
Metre Gauge     int64
Narrow Gauge    int64
Total           int64
dtype: object
```

```
The COLUMN Names
Index(['Year', 'Broad Gauge', 'Metre Gau
ge', 'Narrow Gauge', 'Total'], dtype='st
r')
Year            0
Broad Gauge     0
Metre Gauge     0
Narrow Gauge    0
Total           0
```

Scenario 2: Simple Visualization

1. Extract Year and Columns.

So checking the year in the data set it contains the data like 1965-66 and we have many years. This would lead to congestion in graph. So, I've used ".str[:4]" to extract the only the first 4 characters in the year and it's in string data type , converting it into "int" by using ".astype(int)".

2. Plot Line Graph Showing Total Tracks Over years.

To plot the line graph, first we extract the years into separate variable and then passed for plotting. We for Data Visualization we've used "Matplotlib" and from this we import "pyplot" for plotting. So we have passed the variables to "plt.plot()"-(this plots a line graph).

3. Add Title, X, Y labels.

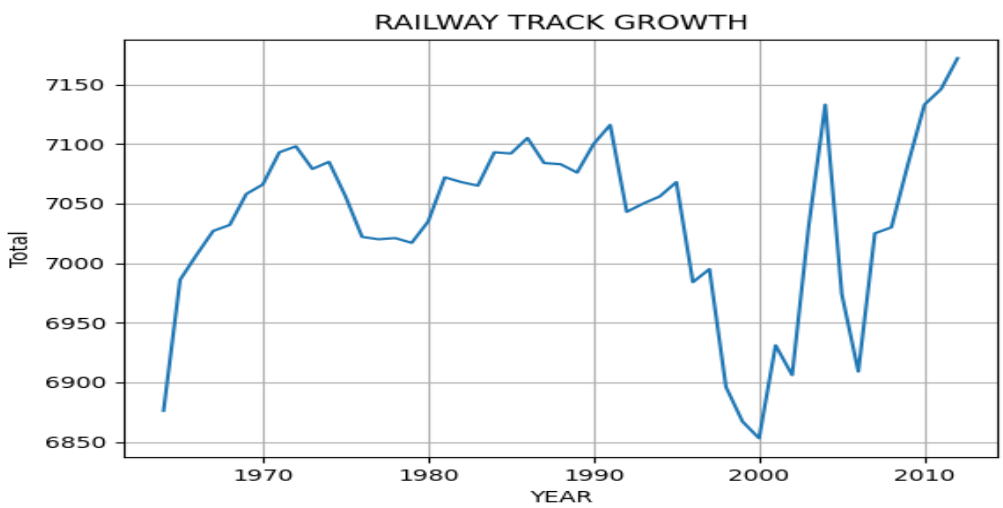
To add the title we have "plt.title()", which adds the title to the line graph and similarly to name X,Y axis we have "plt.xlabel()" and "plt.ylabel()". And it also has a plt.grid() that adds in background for clear and easy to be readable. "plt.Savefig()"-saves the figure and "plt.Show()"—displays the line graph.

```
df['Year'] = df['Year'].str[:4].astype(int)
year=df['Year']
total=df['Total']
plt.plot(year,total)
plt.xlabel("YEAR")
plt.ylabel("Total")
plt.title("RAILWAY TRACK GROWTH")
plt.grid(True)
plt.savefig("trend.png")
plt.show()
```

4. Identify the Trends Increasing / decreasing.

OUTPUT:

From the O/P we can see clearly that the it's increasing, decreasing and then again increasing the growth of the railway track.



Scenario 3: Filtering and Bar Chart

1. Filter the DataSet after Year 2000. Select Broad Gauge, Meter Gauge and Narrow Gauge.

For filtering the data we used pandas DataFrame Operation “df[df[‘Year’]>2000]”. This helps us to get the data or focus on the data that is greater than the year 2000. After that, extracting the data into separate variables like Year, Broad Gauge, metre Gauge and Narrow Gauge. Since we are plotting three bars per year, we use “np.arrange(len(year))” to create evenly spaced positions on the x-axis. Each gauge type is then shifted slightly using offsets (-0.2, 0, +0.2) so the bars come side by side without overlapping.

```
# Filter years after 2000
df['Year_start']=df['Year']
df_recent = df[df['Year'] > 2000]

# Data
year=df_recent['Year']
broad=df_recent['Broad Gauge']
metro=df_recent['Metre Gauge']
narrow=df_recent['Narrow Gauge']

# Positions
x=np.arange(len(year))
```

2. Plot grouped Bar Chart comparing all three gauges.

From the visualizing library we use we use a label from legend that is plt.xticks() that maps the numeric positions to the values of years.

3. Add legend and proper labels

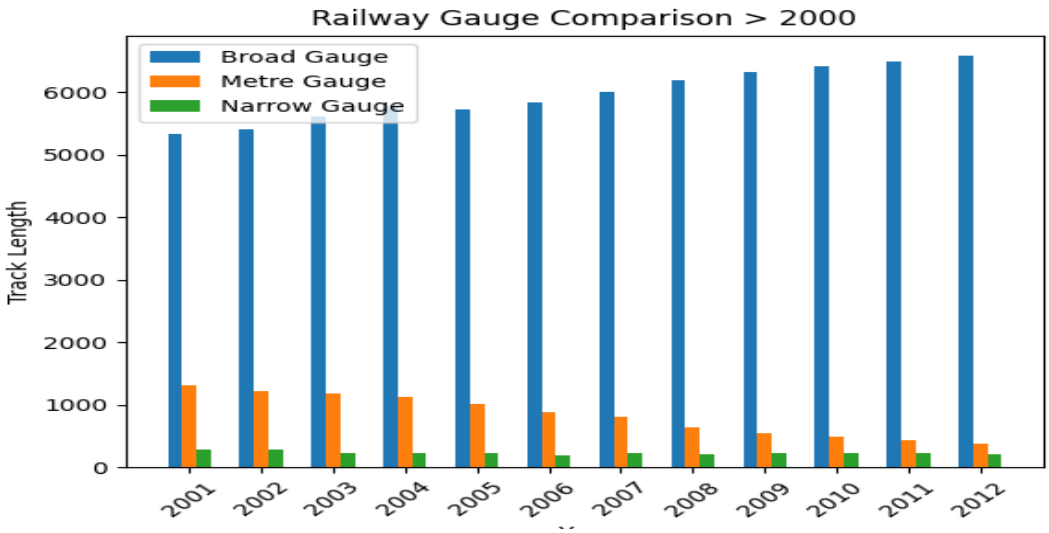
“plt.xlabel()”and “plt.ylabel()”label the x and y axes.“plt.title()” describes the chart “plt.title()” describes the chart “plt.legend()” distinguishes the three gauge types by colour. “plt.savefig('bar.png') used to save the graph.

```
plt.bar(x-0.2,broad,width=0.2,label='Broad Gauge')
plt.bar(x,metro,width=0.2,label='Metre Gauge')
plt.bar(x+0.2,narrow,width=0.2,label='Narrow Gauge')

plt.xticks(x, year, rotation=45)

plt.xlabel("Year")
plt.ylabel("Track Length")
plt.title("Railway Gauge Comparison > 2000")
plt.legend()
plt.savefig('bar.png')#saving the figure ie bar graph
plt.show()
```

Output:



4. Identify which gauge dominates in recent years.

According to the visualization it's clear that Broad gauges Dominates in recent years.

Scenario 4: Featuring and Pie Chart

1. Calculate total sum of each gauge across all years. Create a new structure (Series/DataFrame) for totals.

From pandas we use df.sum() which add up all values in each column in every year. This gives us the total of Broad, Metre, and Narrow gauge. And creating total_df that is a new data frame using "plt.DataFrame()" to get a more precise output .

```
total=df.sum()
#Creating  a new structure for totals
total_df = pd.DataFrame(total, columns=['Total'])
```

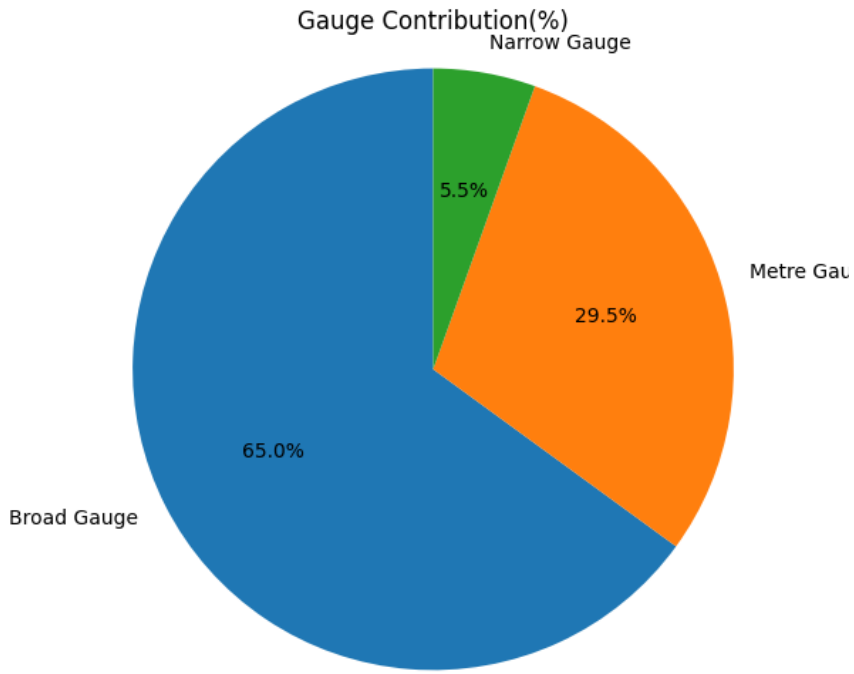
2. Plot a pie chart showing percentage contribution. Add percentage labels (autopct).

The totals are passed into "plt.pie()" to generate a pie chart. Each slice represents one gauge type, and its size reflects its percentage contribution to the overall track length. The "autopct" parameter automatically calculates and displays the percentage on each slice, making it easy to interpret the chart without referring back to the numbers and " plt.axis('equal')" ensures the pie chart is drawn as a perfect circle.

```
plt.pie(total,labels=total.index,autopct='%1.1f%%', startangle=90)
# 4. Add title
plt.title('Gauge Contribution(%)')

# Equal aspect ratio ensures pie is drawn as a circle
plt.axis('equal')
plt.savefig('pie.png')
plt.show()
```

OUTPUT:



3. Interpret which gauge contributes the most.

From the output we can clearly see that Broad Gauge contributes the most.

Scenario 5: Advanced Analysis + Multiple Graphs

1. Create new columns % Broad Gauge, % Metre Gauge%, Narrow Gauge

We create three new columns in the DataFrame, one for each gauge type, using the formula: (Gauge Value / Total) × 100. This tells us what percentage of the total track length each gauge type represents for every year, making comparisons more meaningful than numbers.

```
#we have the formula to convert into percentage as (x/total)*100
df['(%)Broad Gauge']=(df['Broad Gauge']/df['Total'])*100
df['(%)Metre Gauge']=(df['Metre Gauge']/df['Total'])*100
df['(%)Narrow Gauge']=(df['Narrow Gauge']/df['Total'])*100
```

2. Use NumPy (np.diff) to calculate yearly growth of Total tracks.

We use np.diff() from NumPy to calculate the difference between consecutive years in the Total column. This gives us how much the total track length increase or decrease each year. The prepend

argument ensures the first row is not lost, keeping the DataFrame size consistent. "Prepend=df['Total'].iloc[0]", adds the first value at the beginning to maintain the same length.

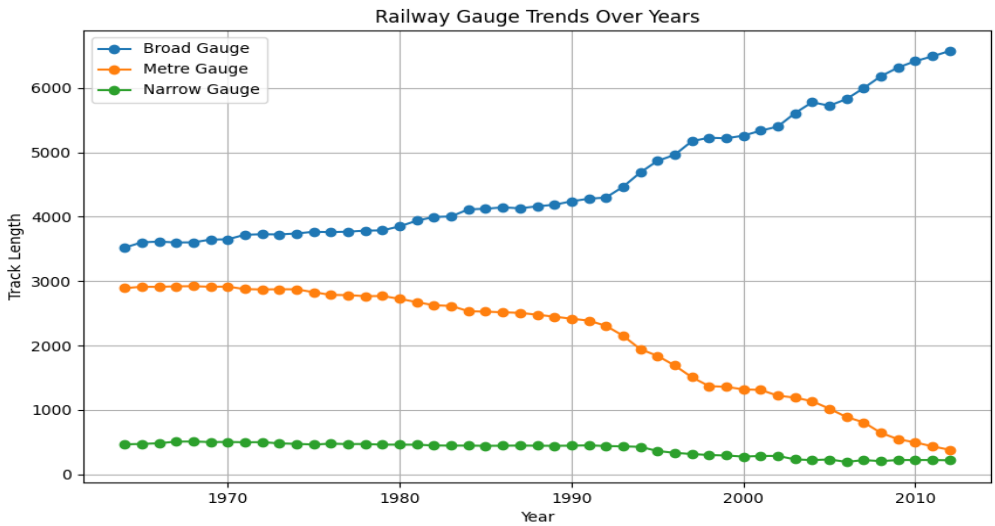
```
#calculating yearly growth
df['Yearly Growth'] = np.diff(df['Total'],prepend=df['Total'].iloc[0])
```

3. Plot Line graph for all gauges, stacked bar chart showing composition, Highlight: Years with highest growth Decline in any gauge

Axes are labelled for clarity, plt.legend() gives the three gauge lines by different colour, plt.grid(True) makes the graph easy to read.

```
plt.legend()
plt.grid(True)
```

OUTPUT:



CONCLUSION

This project gave me a clear idea and understanding of how to work with real world data. I have taken the data analysed it and done visualization in many ways. The data clearly shows that Broad Gauge has dominated and growing, while Metre and Narrow Gauge have declined in the years.

FINAL CONCLUSION

From all the scenarios, I explored the DataSet like loading, comparing and visualising the graphs, and performing analysis with percentage in pie charts and also with line graphs.

WHAT I HAVE LEARNT?

By doing this project I learned how to load and clean a real dataset using Pandas and handle problems like missing values and wrong data types. I understood how NumPy helps in doing quick calculations. I also learned how to create different types of graphs like line graphs, bar charts and pie charts using Matplotlib. Most importantly I learned that data analysis is not just about writing code it is about writing the code + making the visuals that are understood by all.